Othello : comprendre le code et l’algorithme alpha-bêta

Guide de lecture du projet — version du 6 novembre 2026

Ce document décrit la version actuelle du code, après sa simplification. Il explique comment une partie appelle l’IA, comment alpha-bêta choisit un coup et quelles parties du moteur MinMax sont réutilisées. Les exemples se rapportent aux fichiers du projet, sans ajouter de fonctionnalité au programme.

La réponse à la question principale

Oui, alpha-bêta utilise le code de minmax.py. La classe AlphaBetaEngine hérite de MinimaxEngine et importe la fonction evaluate. Elle garde la gestion de la mémoire, du temps et des statistiques du parent, tout en fournissant ses propres méthodes de choix et d’exploration des coups.

Le résultat recherché reste celui de MinMax à profondeur identique. L’élagage évite seulement de calculer certaines branches devenues inutiles pour le choix. Une recherche interrompue par le délai peut atteindre une profondeur différente avec chaque algorithme.

Plan du document

Page	Contenu
2	Organisation du programme et classes du jeu
3	Les classes de recherche et l’héritage de MinimaxEngine
4	Du menu au coup joué : chaîne réelle des appels
5	MinMax, alpha, beta et exemple détaillé d’une coupure
6	Lecture pas à pas de la méthode récursive _alphabeta
7	Mémoire des valeurs exactes et sécurité du cache
8	Choix du coup et fonctions accessibles dans alphabeta.py
9	Délai, approfondissement progressif et statistiques
10	Utilisation, vérifications et limites de cette version

Conventions : BLACK = 1, WHITE = -1, EMPTY = 0. Les Blancs commencent dans ce TP. Les coordonnées Python sont (ligne, colonne), à partir de zéro : e3 correspond à (2, 4).

2. Organisation et classes du jeu

Board : les règles et la position

Board, dans board.py, contient size et grid, un tableau NumPy de pions. reset() place les quatre pions centraux. get_flips() regarde les huit directions : une ligne de pions adverses est capturée si elle est fermée par un pion du joueur. Une case occupée ou sans capture ne constitue pas un coup légal.

get_legal_moves(color) énumère les coups possibles. apply_move(move, color) pose le pion, retourne les captures et les renvoie. copy() produit une grille indépendante : chaque hypothèse de l'IA travaille sur une copie. is_game_over() exige que les deux joueurs soient bloqués ; un seul joueur bloqué doit simplement passer. get_score() compte les pions et get_winner() compare les scores.

Player et ses trois implémentations

Classe	Responsabilité
Player	Classe abstraite : stocke color et name ; impose choose_move(board).
HumanPlayer	Demande une coordonnée via ConsoleView et recommence tant que la saisie est invalide. Renvoie None si aucun coup n'est possible.
RandomPlayer	Choisit au hasard parmi les coups légaux, ou renvoie None.
AIPlayer	Crée un moteur MinimaxEngine ou AlphaBetaEngine, garde depth et time_limit, puis délègue son choix à engine.search().

Deux AIPlayer créent deux moteurs indépendants. Même si les deux utilisent alpha-bêta, leurs dictionnaires de mémoire sont différents. Chaque joueur conserve son propre moteur entre ses tours pour réutiliser les analyses compatibles.

Game, MoveRecord et ConsoleView

Game orchestre la partie : joueur courant, plateau, alternance et historique. play_turn() calcule les coups légaux, donne une copie du plateau au joueur, valide le coup obtenu puis l'applique au vrai plateau. Si aucun coup n'est possible, il enregistre une passe. run() répète les tours jusqu'à la fin et renvoie le gagnant.

MoveRecord est une dataclass contenant color, move et flipped. move=None représente une passe. save_history() exporte cet historique en texte pour un fichier .txt, sinon en JSON. ConsoleView affiche le plateau, les coups et le score ; parse_move() et format_move() convertissent les coordonnées. Elle ne décide pas des règles ni du choix de l'IA.

3. Classes de recherche et héritage

```
from minmax import MinimaxEngine, evaluate

class AlphaBetaEngine(MinimaxEngine):
    ...
```

Cette déclaration signifie qu'un AlphaBetaEngine possède les méthodes et attributs de MinimaxEngine, sauf lorsqu'il redéfinit une méthode. Aucun deuxième moteur caché n'est créé : self désigne toujours le même objet AlphaBetaEngine, y compris dans une méthode héritée.

Élément	Origine et rôle dans alpha-bêta
__init__()	Hérité : initialise table, max_entries, deadline, stats et last_result.
clear()	Hérité : vide la table et remet à zéro l'état des mesures.
_check_time()	Hérité : compare perf_counter() à deadline et peut lever SearchTimeout.
search()	Hérité : valide les paramètres, prépare le délai, organise les profondeurs et construit SearchResult.
_best_move()	Redéfini : choisit un coup en réutilisant le meilleur score comme alpha.
_minimax()	Redéfini : ouvre une fenêtre complète et appelle _alphabeta().
_alphabeta()	Ajouté : réalise la récursion et les coupures alpha-bêta.
evaluate()	Fonction importée, pas méthode héritée : compte l'avance en pions.

Les objets de résultat, définis dans minmax.py

SearchStats est une dataclass avec nodes, cache_hits, completed_depth, elapsed_seconds et timed_out. Une nouvelle instance est créée pour chaque décision ; la table de mémoire, elle, reste conservée.

SearchResult regroupe move, score et stats. Un score None signifie qu'aucune valeur n'a été calculée pour le choix retourné. Ce n'est ni zéro ni une égalité. move=None signifie qu'il n'y a aucun coup légal à proposer.

SearchTimeout est une exception interne. _check_time() la lève quand le délai est dépassé ; search() la capture pour renvoyer un choix exploitable au lieu d'arrêter la partie.

Point important : la classe TableEntry et les étiquettes EXACT, LOWER, UPPER n'existent plus dans la version simplifiée. table associe directement une clé à un score exact pour une profondeur donnée.

4. Du menu au coup réellement joué

La chaîne d’appels normale

```
main()
  -> choose_player()
  -> Game.run()
     -> Game.play_turn()
        -> AIPlayer.choose_move(copie_du_plateau)
           -> engine.search(...)        # code de MinimaxEngine
              -> self._best_move(...)   # code de AlphaBetaEngine
                 -> self._alphabeta(...)
                    -> self._alphabeta(...)  # appels recursifs
```

1. Configuration. main.py demande séparément les joueurs noir et blanc. Le choix alpha-bêta entraîne AIPlayer(..., algorithm="alphabeta"). Le dictionnaire engines sélectionne AlphaBetaEngine, puis son constructeur hérité initialise les attributs.

2. Début d’un tour. Game vérifie que la partie n’est pas finie. Si le joueur peut agir, il reçoit une copie du plateau. AIPlayer.choose_move() appelle self.engine.search(board, self.color, self.depth, self.time_limit).

3. Recherche. Python trouve search() dans le parent MinimaxEngine. À l’intérieur, l’appel self._best_move(...) cherche la méthode sur l’objet réel : il exécute donc la version redéfinie dans AlphaBetaEngine. C’est le mécanisme de résolution des méthodes, souvent appelé polymorphisme.

4. Retour. search() produit un SearchResult. AIPlayer le conserve dans last_result, puis renvoie seulement result.move. Game vérifie ce coup et applique une seule fois le changement sur le plateau réel, enregistre un MoveRecord et change de joueur.

Ce qui n’est pas exécuté

Lors d’un choix alpha-bêta, on n’exécute pas d’abord une recherche MinMax complète. Les versions du parent _best_move() et _minimax() ne sont pas utilisées par ce chemin : elles sont remplacées par les méthodes de la classe enfant. Les outils communs de gestion sont réutilisés, pas l’exploration exhaustive du parent.

La méthode _minimax() de la classe enfant sert surtout d’entrée pour évaluer directement une position avec la fonction alphabeta(). Le choix normal des coups passe directement de _best_move() à _alphabeta(). Le fichier search.py n’est pas utilisé par ce chemin du jeu.

5. Principe de MinMax et alpha-bêta

Un score toujours vu par la même IA

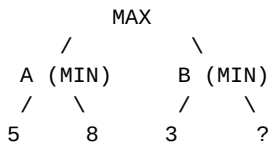
evaluate(board, ai_color) calcule count(ai_color) − count(-ai_color). Par exemple, 10 pions de l'IA contre 6 donnent +4. ai_color reste fixe pendant toute la recherche. current_color change à chaque tour : si current_color == ai_color, le nœud maximise ; sinon il minimise. Le rôle MAX dépend donc de l'IA, pas de la couleur noire ou blanche.

MinMax simule les coups et suppose que l'adversaire répond au mieux pour lui. À un nœud MAX, on prend le maximum des enfants ; à un nœud MIN, leur minimum. La limite depth interrompt la simulation et utilise evaluate() si la partie n'est pas déjà terminée.

À quoi servent alpha et beta ?

alpha est la borne basse transmise par les choix déjà disponibles à MAX. **beta** est la borne haute transmise par les choix déjà disponibles à MIN. MAX peut relever alpha ; MIN peut abaisser beta. Dès que alpha >= beta, les autres enfants de ce nœud ne peuvent plus améliorer la décision pertinente plus haut : on arrête leur exploration.

Exemple chiffré, indépendant d'une grille particulière




```
A vaut min(5, 8) = 5. MAX dispose donc de alpha = 5.
Dans B, la premiere reponse vaut 3 : beta devient 3.
alpha >= beta : 5 >= 3, on ne calcule pas le ?.
```

Même si la feuille inconnue valait 100, MIN choisirait 3. Si elle valait -10, MIN choisirait -10. Dans les deux cas, B ne peut pas battre le 5 déjà disponible avec A. La coupure ne suppose donc pas la valeur de la feuille ignorée : elle prouve que cette valeur est inutile pour ce choix.

L'ordre des coups influence le nombre de coupures. Ici, les coups restent dans l'ordre fourni par Board.get_legal_moves(). Le programme n'ajoute pas de tri heuristique. L'élagage peut fortement réduire l'exploration, mais ne garantit pas le même gain sur toutes les positions.

6. Lecture de _alphabeta(), étape par étape

Paramètre	Signification
board	Position simulée ; elle est copiée avant chaque coup enfant.
depth	Nombre de demi-coups restant à explorer.
current_color	Joueur qui doit agir dans cette position.
ai_color	Point de vue fixe pour tous les scores.
alpha, beta	Fenêtre utile transmise par les appels précédents.

1. Délai et mémoire. La méthode appelle _check_time(), incrémente nodes et construit la clé. Si la table contient déjà une valeur pour cette clé, elle incrémente cache_hits et la retourne immédiatement.

2. Fenêtre initiale. initial_alpha et initial_beta gardent les bornes à l'entrée de l'appel. Elles serviront à savoir si le score final peut être mémorisé comme exact. leaf commence à False.

3. Fin de partie. is_game_over() est testé avant depth == 0. Une victoire de l'IA vaut +inf, une défaite -inf, une égalité 0. Toutes les victoires certaines sont donc équivalentes, quel que soit l'écart final de pions. Ce test met leaf à True.

4. Limite de profondeur. Si la partie continue mais depth vaut 0, evaluate() renvoie la différence de pions, et leaf devient True. Cette évaluation est exacte pour l'heuristique choisie, pas une garantie sur le gagnant final.

5. Passe. Si le joueur n'a aucun coup légal, la méthode rappelle _alphabeta() avec le même plateau et la même profondeur, mais -current_color. Une passe ne consomme pas de demi-coup. Si les deux joueurs étaient bloqués, le test de fin aurait déjà arrêté la récursion.

6. Exploration. Pour chaque coup, la méthode copie board, applique le coup et descend avec depth - 1 et le joueur opposé. MAX met à jour score=max(score, value), puis alpha=max(alpha, score). MIN utilise min et met beta à jour. Si alpha >= beta, break arrête la boucle des enfants de cet appel seulement.

7. Retour et cache. Un dernier contrôle du temps précède l'enregistrement. Seule une feuille directement évaluée ou une valeur strictement à l'intérieur de la fenêtre initiale est enregistrée. La méthode renvoie ensuite score : selon la fenêtre, ce retour peut être exact ou n'être qu'une borne suffisante pour le parent.

7. La mémoire simplifiée : valeurs exactes

Une clé complète pour éviter les confusions

```
key = (board.size, board.to_key(),
       current_color, ai_color, depth)

if key in self.table:
    self.stats.cache_hits += 1
    return self.table[key]
```

to_key() transforme la grille en octets utilisables comme clé. La taille, le joueur au trait, le point de vue et la profondeur font également partie de la clé. La même grille évaluée à deux profondeurs différentes peut avoir deux scores différents ; les entrées ne doivent donc pas être confondues.

Pourquoi ne pas mémoriser chaque retour ?

Dans l'exemple précédent, B retourne 3 après avoir vu une seule réponse. Sa vraie valeur pourrait être -10 si un enfant non exploré valait -10. Le nombre 3 n'est alors qu'une borne supérieure : la valeur réelle est au plus 3. Le mettre en cache comme valeur exacte fausserait une future recherche.

```
if leaf or initial_alpha < score < initial_beta:
    if len(self.table) >= self.max_entries and key not in self.table:
        del self.table[next(iter(self.table))]
    self.table[key] = score
```

La version actuelle utilise une règle conservatrice : enregistrer les feuilles, dont la valeur a été calculée directement, et les résultats internes strictement compris entre les bornes d'entrée. Un résultat inférieur ou égal à alpha initial, ou supérieur ou égal à beta initial, n'est pas enregistré comme résultat interne.

Tester seulement l'absence de break dans le nœud courant ne suffirait pas : un enfant peut lui-même renvoyer une borne. C'est pourquoi on conserve et compare la fenêtre initiale. Certaines valeurs pourtant exactes, notamment des infinis remontés depuis des enfants, peuvent être omises ; cela coûte des recalculs mais ne rend pas le choix incorrect.

Durée de vie et taille

Le parent initialise max_entries à 100 000. Quand une nouvelle clé dépasserait la capacité, la première clé du dictionnaire est supprimée : c'est l'entrée la plus ancienne par insertion, pas la moins récemment consultée. clear() vide explicitement le cache. search() remet les statistiques à zéro mais conserve les entrées pour les prochains tours.

La réutilisation au tour suivant exige la même position, le même trait, le même point de vue et la même profondeur restante. Par exemple, une analyse à profondeur 4 peut contenir des positions à profondeur restante 2 après deux coups ; elles sont réutilisables pour une recherche ultérieure à ce même horizon.

8. Choisir un coup et utiliser les fonctions

_best_move() : la racine de la décision

search() lui fournit une liste non vide de coups légaux. Elle initialise best_move au premier coup et best_score à -inf. Ce choix initial garantit qu'une défaite forcée renvoie tout de même un coup légal : si tous les scores valent -inf, la condition score > best_score ne sera jamais vraie.

```
child = board.copy()
child.apply_move(move, ai_color)
score = self._alphabeta(
    child, depth - 1, -ai_color, ai_color,
    best_score, float("inf")
)
```

Le premier coup vient d'être joué : il consomme un niveau et donne le trait à l'adversaire. La fenêtre commence à best_score, car l'IA dispose déjà de ce résultat avec un autre coup. Si le candidat ne peut pas faire mieux, l'adversaire peut provoquer une coupure. Si score est strictement meilleur, le coup et son score deviennent les nouveaux meilleurs.

À égalité, le premier coup reste choisi. Si best_score atteint +inf, une victoire certaine est trouvée : aucune valeur ne peut l'améliorer, donc la boucle s'arrête. La méthode contrôle encore le délai, puis renvoie le couple (best_move, best_score).

_minimax() et alphabeta() : évaluer seulement

La méthode _minimax() de AlphaBetaEngine sert de passerelle : elle appelle _alphabeta() avec alpha=-inf et beta=+inf. Elle ne lance pas la récursion MinMax du parent. La fonction alphabeta(board, depth, current_color, ai_color) vérifie que depth est un entier non négatif, crée un moteur neuf et utilise cette passerelle.

alphabeta() renvoie un score, pas un coup. Elle accepte depth=0 et n'impose pas de délai. Le moteur créé à chaque appel n'est pas conservé pour l'appel suivant ; cette fonction est surtout pratique pour des évaluations directes ou des tests.

choose_alphabeta_move() : obtenir seulement le coup

Cette fonction crée un moteur si engine n'est pas fourni, puis renvoie engine.search(...).move. Fournir le même moteur permet de garder la mémoire. Pour obtenir également les statistiques et le score, il vaut mieux appeler engine.search() directement. Aucun coup légal donne None, que la partie soit terminée ou que le joueur doive passer.

```
engine = AlphaBetaEngine()
result = engine.search(board, WHITE, depth=4, time_limit=2.0)
move = result.move
score = result.score
nodes = result.stats.nodes
```

9. Temps, profondeur et statistiques

La profondeur du code et celle du TP

Un demi-coup est l'action d'un joueur. depth=2 représente un coup de l'IA puis une réponse adverse, soit une profondeur de 1 selon la convention du TP. Dans _best_move(), le coup initial consomme déjà un niveau. Une passe ne réduit pas depth.

search() : une méthode héritée essentielle

Sans délai (time_limit=None), search() lance directement la profondeur demandée. Avec un délai, elle calcule successivement les niveaux 1, 2, 3… jusqu'au maximum. Ce procédé donne rapidement un premier choix complet, puis tente de l'améliorer. Les compteurs cumulent le travail de ces différentes itérations.

Le délai est converti en échéance : deadline = started + time_limit. perf_counter() mesure une durée. Les appels à _check_time() interrompent la recherche par SearchTimeout. Le contrôle est coopératif : une opération du plateau déjà commencée se termine avant le prochain contrôle, donc un léger dépassement reste possible.

Le nouveau meilleur coup n'est publié qu'après le retour réussi de _best_move(). Si le niveau 3 est interrompu, le niveau 2 reste utilisable. Les feuilles et sous-arbres exacts déjà terminés peuvent rester en mémoire ; un nœud interrompu n'atteint pas son enregistrement final. Si aucun niveau ne finit, le premier coup légal sert de secours, avec score=None et completed_depth=0.

Champ de SearchStats	Interprétation
nodes	Nombre d'appels récursifs, y compris ceux résolus par le cache. La boucle racine _best_move n'est pas elle-même comptée comme un nœud.
cache_hits	Nombre de valeurs renvoyées directement depuis la mémoire.
completed_depth	Dernière profondeur entièrement résolue ; des coupures valides n'empêchent pas un niveau d'être terminé.
elapsed_seconds	Temps de la décision, y compris préparation et recherche.
timed_out	True si SearchTimeout a interrompu l'exploration.

Attention à la valeur zéro : dans le menu, 0 seconde est converti en None et signifie « sans limite ». Dans l'API Python, time_limit=0 signifie réellement un budget nul et déclenche normalement le choix de secours. Sans coup légal, aucune recherche n'est nécessaire : move et score valent None.

10. Utilisation, validation et limites

Lancer une partie ou une comparaison

```
.\.venv\Scripts\python.exe main.py
.\.venv\Scripts\python.exe -m pytest -q
```

Le menu propose humain, MinMax, alpha-bêta ou aléatoire pour chaque couleur. Pour comparer les moteurs à horizon identique, choisir la même profondeur et un temps de 0 dans le menu, donc sans limite. Pour comparer à budget de temps identique, choisir le même délai et observer aussi la profondeur effectivement atteinte.

Vérification sur la version documentée

Les **33 tests passent** au moment de la création de ce document. Ils couvrent les règles du plateau, les parties entre IA, les choix du menu, la comparaison avec MinMax, les passages de tour, les résultats terminaux, le cache et les délais simulés. Les petits plateaux 4×4 accélèrent certaines vérifications ; le jeu normal utilise 8×8.

Position initiale 8×8, Blanc, profondeur 4, sans délai	MinMax	Alpha-bêta
Coup choisi	e3 (2, 4)	e3 (2, 4)
Score	-2	-2
Appels récursifs (nodes)	312	132
Accès utiles au cache	6	0

Mesure obtenue avec deux moteurs neufs sur le même plateau. Alpha-bêta explore ici environ 58 % de nœuds en moins. Ce résultat illustre une position précise ; il ne constitue pas une garantie de gain constant. Les tests de mémoire vérifient aussi la réutilisation lors d'appels ultérieurs.

Ce que cette version ne promet pas

L'évaluation par nombre de pions est simple : une avance en cours de partie n'assure pas la victoire. La profondeur limitée peut masquer une menace plus lointaine. Le moteur n'ordonne pas les coups par intérêt stratégique. Le cache exact seulement est plus facile à comprendre, mais peut recalculer des branches qu'un cache de bornes plus complexe aurait évitées.

Les infinis classent les victoires et défaites certaines ; ils ne départagent pas deux victoires par leur marge. Deux moteurs disposant du même temps peuvent atteindre des horizons différents. Enfin, les tests établissent des vérifications concrètes sur plusieurs cas, pas une preuve exhaustive de toutes les parties possibles.

Fichiers de référence dans le projet

Analyse fondée sur **alphabeta.py, minmax.py, board.py, constants.py, player.py, game.py, view.py et main.py**, ainsi que tests/test_alphabeta.py, tests/test_minmax.py, tests/test_board.py et tests/test_game.py. Le document décrit le code présent, notamment le cache simplifié sans TableEntry ; il ne décrit pas une version antérieure.